

循序渐进，学习开发 xv6-riscv

第 3 章 Hello xv6

汪辰



目录

01

UART 简介

02

NS16550A

03

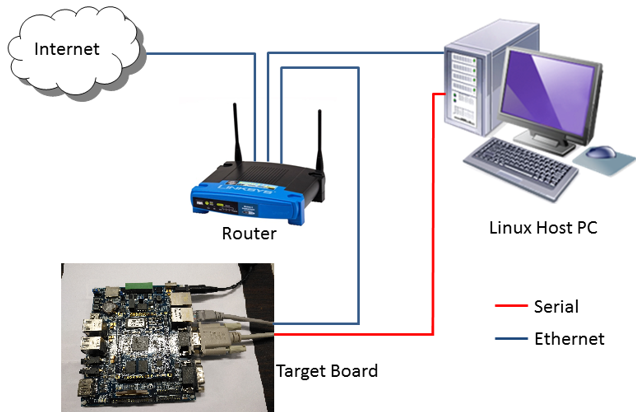
利用串口实现打印输出



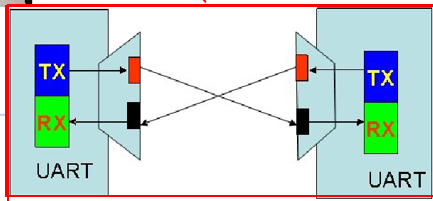
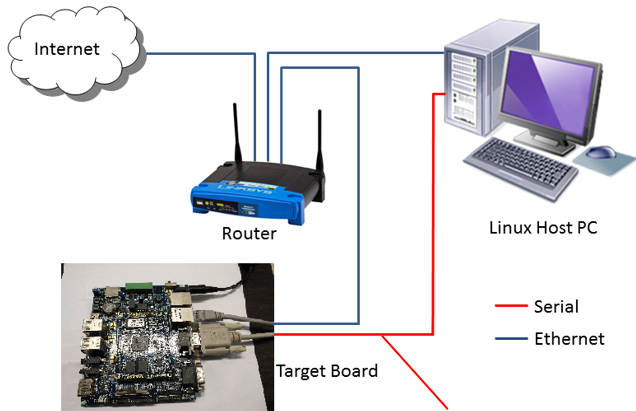
01

UART 简介

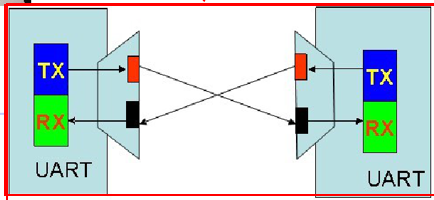
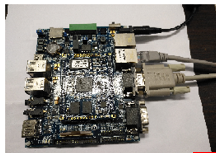
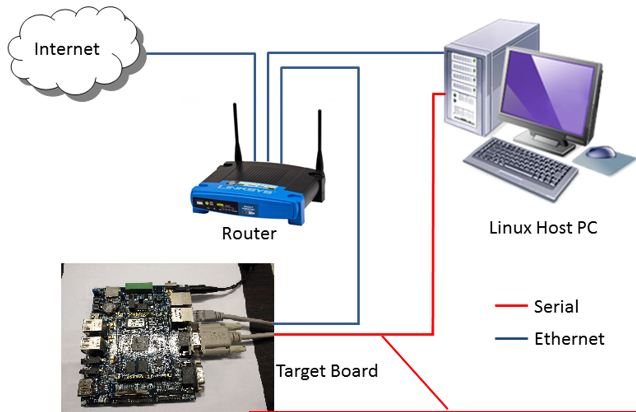
UART 的硬件连接方式



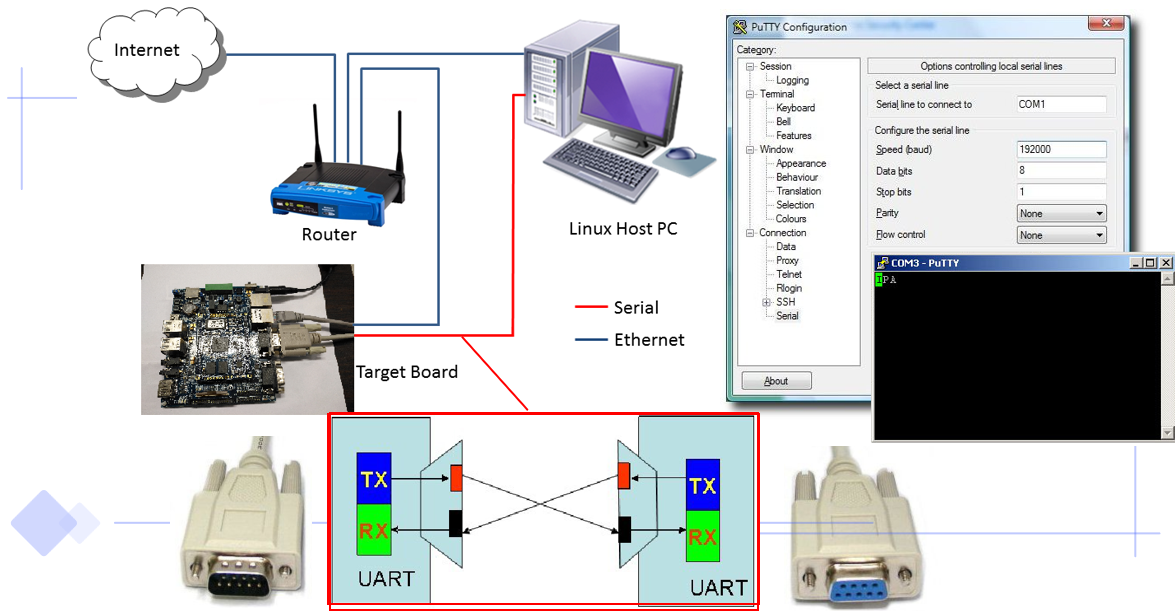
UART 的硬件连接方式



UART 的硬件连接方式



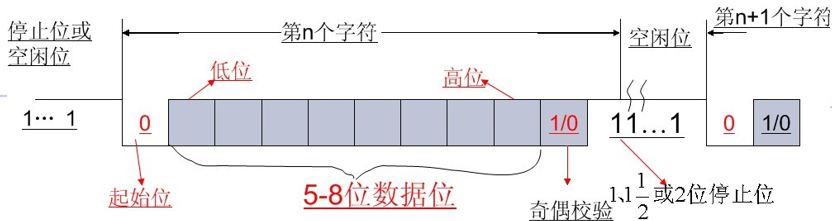
UART 的硬件连接方式



UART 的特点

- UART (**U**niversal **A**synchronous **R**eceiver and **T**ransmitter)
- **串行**：相对于并行，串行是按位来进行传递，即一位一位的发送和接收。**波特率(baud rate)**，每秒传输的二进制位数，单位为 bps(bit per second)。
- **全双工**：相对于单工和半双工，全双工指可以同时进行收发两方向的数据传递。
- **异步**：相对于同步，异步数据传输的过程中，不需要时钟线，直接发送数据，但需要约定通讯协议格式。

UART 的通讯协议



- **空闲位**：总线处于空闲状态时信号线的状态为 '1' 即高电平。
- **起始位**：发送方要先发出一个低电平 '0' 来表示传输字符的开始。
- **数据位**：起始位之后就是要传输的数据，数据长度（word length）可以是 5/6/7/8/9 位，构成一个字符，一般都是 8 位。先发送最低位最后发送最高位。
- **奇偶校验位（parity）**：串口校验分几种方式：
 - ✓ 无校验（no parity）
 - ✓ 奇校验（odd parity）：确保传输数据（包含校验位）中1的个数为奇数。即：如果传输数据位中1的个数是偶数，则校验位取值为“1”（补足使得总的1的个数为奇数），否则如果传输数据位1的个数是奇数，则校验位取值为“0”。
 - ✓ 偶校验（even parity）：确保传输数据（包含校验位）中1的个数为偶数。即：如果传输数据位中1的个数是奇数，则校验位取值为“1”（补足使得总的1的个数为偶数），否则如果传输数据位1的个数是偶数，则校验位取值为“0”。
 - ✓ mark parity：校验位始终为1
 - ✓ space parity：校验位始终为0
- **停止（stop）位**：数据结束标志，可以是 1位，1.5 位，2 的高电平。

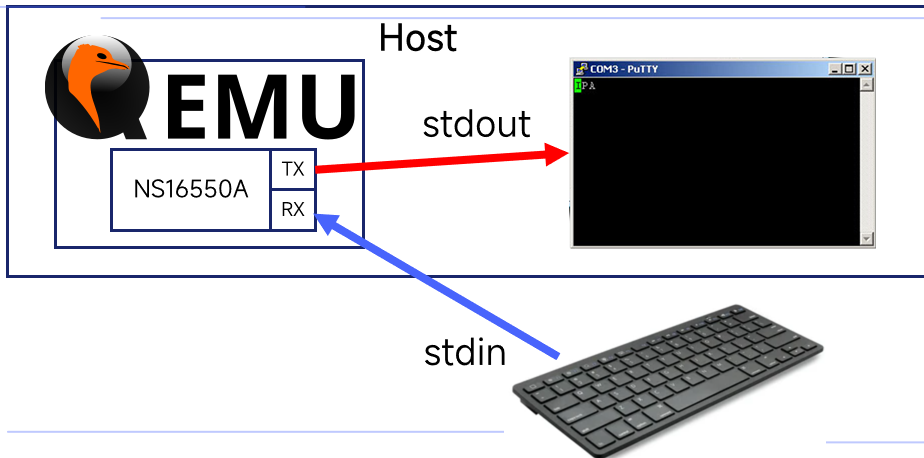


02

NS16550A

NS16550a 编程接口介绍

QEMU 的 virt 机器默认模拟的 UART 控制器型号是 NS16550A。



NS16550a 编程接口介绍

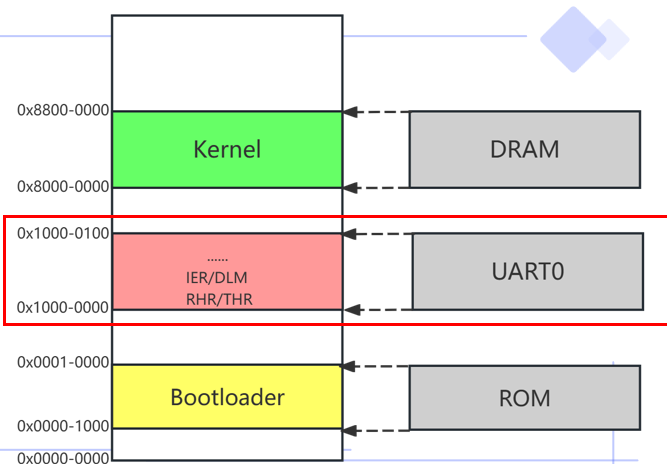
QEMU 的 virt 机器模拟的 NS16550A 包含以下典型特征:

- 支持基于内存映射 I/O。
- 实现一套广泛兼容的标准寄存器集。通过这些寄存器，软件可以配置串口参数（如波特率）、发送和接收数据，以及处理中断。
- FIFO 支持：模拟了 16550A 的 16 字节发送和接收 FIFO。
- 支持中断方式读写，即当数据到达或发送缓冲区为空时产生中断。

NS16550a 编程接口介绍

<https://github.com/qemu/qemu/blob/master/hw/riscv/virt.c>

```
static const MemMapEntry virt_memmap[] = {  
    [VIRT_DEBUG] = { 0x0, 0x100 },  
    [VIRT_MROM] = { 0x1000, 0xf000 },  
    [VIRT_TEST] = { 0x100000, 0x1000 },  
    [VIRT_RTC] = { 0x101000, 0x1000 },  
    [VIRT_CLINT] = { 0x2000000, 0x10000 },  
    [VIRT_ACLINT_SSWI] = { 0x2f00000, 0x4000 },  
    [VIRT_PCIE_PIO] = { 0x3000000, 0x10000 },  
    [VIRT_IOMMU_SYS] = { 0x3010000, 0x1000 },  
    [VIRT_PLATFORM_BUS] = { 0x4000000, 0x2000000 },  
    [VIRT_PLIC] = { 0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },  
    [VIRT_APLIC_M] = { 0xc000000, APLIC_SIZE(VIRT_CPUS_MAX) },  
    [VIRT_APLIC_S] = { 0xd000000, APLIC_SIZE(VIRT_CPUS_MAX) },  
    [VIRT_UART0] = { 0x10000000, 0x100 },  
    [VIRT_VIRTIO] = { 0x10001000, 0x1000 },  
    [VIRT_FW_CFG] = { 0x10100000, 0x18 },  
    [VIRT_FLASH] = { 0x20000000, 0x4000000 },  
    [VIRT_IMSIC_M] = { 0x24000000, VIRT_IMSIC_MAX_SIZE },  
    [VIRT_IMSIC_S] = { 0x28000000, VIRT_IMSIC_MAX_SIZE },  
    [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },  
    [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },  
    [VIRT_DRAM] = { 0x80000000, 0x0 },  
};
```



NS16550a 编程接口介绍

A2	A1	A0	READ MODE	WRITE MODE
0	0	0	• Receive Holding Register	• Transmit Holding Register
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• Interrupt Enable Register
0	0	1	N/A	• MSB of Divisor Latch when Enabled
0	1	0	• Interrupt Status Register	• FIFO control Register
0	1	1	N/A	• Line Control Register
1	0	0	N/A	• Modem Control Register
1	0	1	• Line Status Register	N/A
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read	• Scratchpad Register Write

<http://byterunner.com/16550.html>

NS16550a 编程接口介绍

A2	A1	A0	READ MODE	WRITE MODE
0	0	0	• Receive Holding Register	• Transmit Holding Register
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• Interrupt Enable Register
0	0	1	N/A	• MSB of Divisor Latch when Enabled
0	1	0	• Interrupt Status Register	• FIFO control Register
0	1	1	<pre>// the UART control registers are memory-mapped // at address UART0. this macro returns the // address of one of the registers. #define Reg(reg) ((volatile unsigned char *)(UART0 + (reg)))</pre>	
1	0	0		
1	0	1	•	
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read	• Scratchpad Register Write

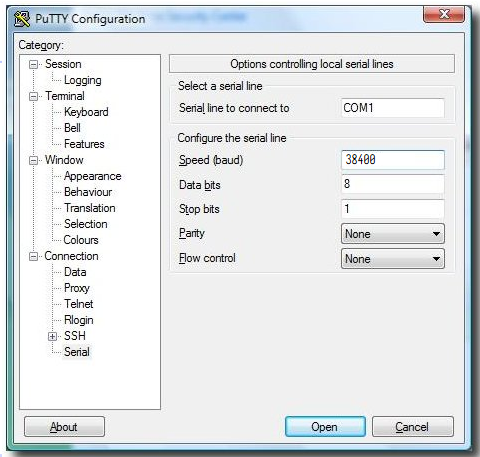
<http://byterunner.com/16550.html>



03

利用串口实现打印输出

串口的初始化



```
void
uartinit(void)
{
    // disable interrupts.
    WriteReg(IER, 0x00);

    // special mode to set baud rate.
    WriteReg(LCR, LCR_BAUD_LATCH);

    // LSB for baud rate of 38.4K.
    WriteReg(0, 0x03);

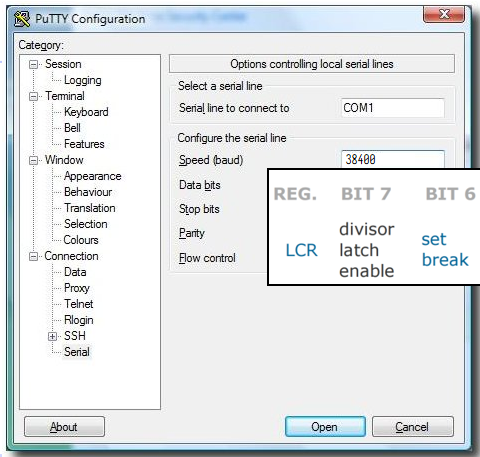
    // MSB for baud rate of 38.4K.
    WriteReg(1, 0x00);

    // leave set-baud mode,
    // and set word length to 8 bits, no parity.
    WriteReg(LCR, LCR_EIGHT_BITS);

    // reset and enable FIFOs.
    WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);
}
```

- IER (Interrupt Enable Register, 中断使能寄存器)
- 写入 0x00 表示关闭所有中断，初始化期间暂不需要中断。

串口的初始化



```
void  
uartinit(void)  
{
```

LCR (Line Control Register, 线路控制寄存器), 用于设置数据格式和波特率。

```
// disable  
WriteReg(IER, 0);
```

```
// special mode to set baud rate
```

```
WriteReg(LCR, LCR_BAUD_LATCH);
```

```
#define LCR_BAUD_LATCH (1<<7)
```

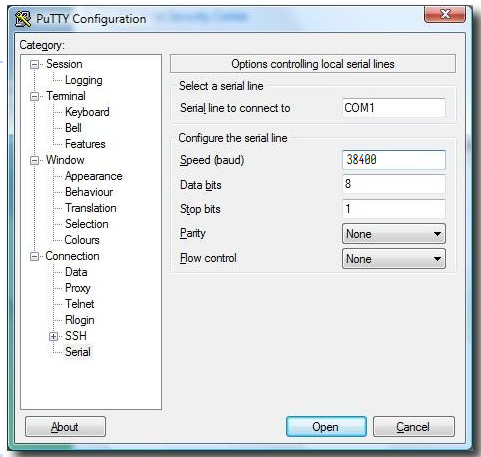
REG.	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LCR	divisor latch enable	set break	set parity	even parity	parity enable	stop bits	word length bit 1	word length bit 0

```
WriteReg(1, 0x00);
```

```
// leave set-baud mode,  
// and set word length to 8 bits, no parity.  
WriteReg(LCR, LCR_EIGHT_BITS);
```

```
// reset and enable FIFOs.  
WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);  
}
```

串口的初始化



```
void  
uartinit(void)  
{
```

```
// disable interrupts.  
WriteReg(IER, 0x00);
```

```
// special mode to s  
WriteReg(LCR, LCR_BA
```

```
// LSB for baud rate of 2  
WriteReg(0, 0x03);  
  
// MSB for baud rate of 38.4K.  
WriteReg(1, 0x00);
```

- 波特率 = 基准时钟频率 / (除数 × 16)
- 基准时钟频率 = 1.8432 MHz
- 除数 (Divisor) = (MSB << 8) | LSB

```
// leave  
// and s  
WriteReg
```

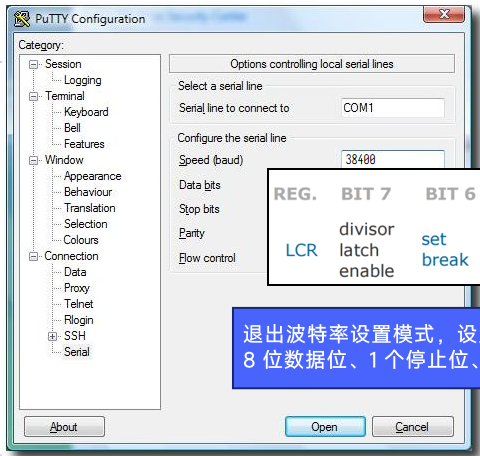
```
// reset
```

```
WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);
```

```
}
```

A2 A1 A0			READ MODE	WRITE MODE
0	0	0	• Receive Holding Register	• Transmit Holding Register
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• Interrupt Enable Register
0	0	1	N/A	• MSB of Divisor Latch when Enabled

串口的初始化



退出波特率设置模式，设置数据格式
8 位数据位、1 个停止位、无校验位

```
void
uartinit(void)
{
    // disable interrupts.
    WriteReg(IER, 0x00);

    // special mode to set baud rate.
    WriteReg(LCR, LCR_BAUD_LATCH);
```

REG.	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0
LCR	divisor latch enable	set break	set parity	even parity	parity enable	stop bits	word length bit 1	word length bit 0

```
WriteReg(1, 0x00);

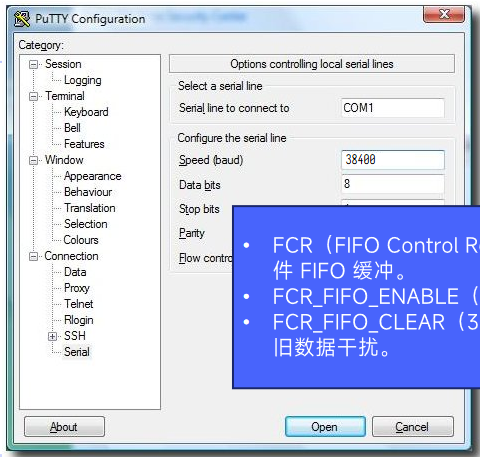
// we set-baud mode,
// set word length to 8 bits, no parity.
```

```
WriteReg(LCR, LCR_EIGHT_BITS);
```

```
#define LCR_EIGHT_BITS (3<<0)
```

```
// reset and enable FIFOs.
WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);
}
```

串口的初始化



- FCR (FIFO Control Register, FIFO 控制寄存器)：控制硬件 FIFO 缓冲。
- FCR_FIFO_ENABLE (1<<0)：启用发送和接收 FIFO。
- FCR_FIFO_CLEAR (3<<1)：清空接收和发送 FIFO，避免旧数据干扰。

```
void
uartinit(void)
{
    // disable interrupts.
    WriteReg(IER, 0x00);

    // special mode to set baud rate.
    WriteReg(LCR, LCR_BAUD_LATCH);

    // LSB for baud rate of 38.4K

    // ... , no parity.
    WriteReg(LCR, LCR_RIGHT_BITS);

    // reset and enable FIFOs.
    WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);
}
```

基于轮询方式实现串口打印输出

A2	A1	A0	READ MODE	WRITE MODE
0	0	0	• Receive Holding Register	• Transmit Holding Register
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• Interrupt Enable Register
0	0	1	N/A	• MSB of Divisor Latch when Enabled
0	1	0	• Interrupt Status Register	• FIFO control Register
0	1	1	N/A	• Line Control Register
1	0	0	N/A	• Modem Control Register
1	0	1	• Line Status Register	N/A
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read	• Scratchpad Register Write

基于轮询方式实现串口打印输出

A2	A1	A0	READ MODE	WRITE MODE
0	0	0	• Receive Holding Register	• Transmit Holding Register
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• Interrupt Enable Register
0	0	1	N/A	• MSB of Divisor Latch when Enabled
0	1	0	• Interrupt Status Register	• FIFO control Register
0	1	1	N/A	• Line Control Register
1	0	0	N/A	• Modem Control Register
1	0	1	• Line Status Register	N/A
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read	• Scratchpad Register Write

- UART 工作方式为全双工，分发送（TX）和接收（RX）两个独立的方向进行数据传输。
- 对数据的 TX/RX 有两种处理方式：
 - ✓ 轮询处理方式
 - ✓ 中断处理方式

基于轮询方式实现串口打印输出

A2	A1	A0	READ MODE	WRITE MODE
0	0	0	• Receive Holding Register	• Transmit Holding Register
0	0	0	N/A	• LSB of Divisor Latch when Enabled
0	0	1	N/A	• Interrupt Enable Register
0	0	1	N/A	• MSB of Divisor Latch when Enabled
0	1	0	• Interrupt Status Register	• FIFO control Register
0	1	1	N/A	• Line Control Register
1	0	0	N/A	• Modem Control Register
1	0	1	• Line Status Register	N/A
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read	• Scratchpad Register Write

- UART 工作方式为全双工，分发送（TX）和接收（RX）两个独立的方向进行数据传输。
- 对数据的 TX/RX 有两种处理方式：
 - ✓ 轮询处理方式
 - ✓ 中断处理方式

```
void
uartputc_sync(int c)
{
    // wait for Transmit Holding Empty to be set in LSR.
    while((ReadReg(LSR) & LSR_TX_IDLE) == 0)
        ;
    WriteReg(THR, c);
}
```


基于轮询方式实现串口打印输出

```
void
main()
{
    uartinit();
    printfinit();
    printf("\n");
    printf("xv6 kernel is booting\n");
    printf("\n");
    while(1);
}
```

1	0	1	• Line Status Register	N/A
1	1	0	• Modem Status Register	N/A
1	1	1	• Scratchpad Register Read • Scratchpad Register Write	

WRITE MODE

Transmit Holding Register

Transmit Latch when Enabled

Transmit Enable Register

Transmit Latch when Enabled

Transmit Register

Transmit Register

Transmit Register

printf(char *fmt, ...)

两种方式
✓ 中断处理方式

```
void
uartputc_sync(int c)
{
    // wait for Transmit Holding Empty to be set in LSR.
    while((ReadReg(LSR) & LSR_TX_IDLE) == 0)
        ;
    WriteReg(THR, c);
}
```

- UART 工作方式为全双工，分发送 (TX) 和接收 (RX) 两个独立的方向进行数据传输。

- 对数据的 TX/RX 有两种处理方式：

基于轮询方式实现串口打印输出

```
void
main()
{
    uartinit();
    printfinit();
    printf("\n");
    printf("xv6 kernel is booting\n");
    printf("\n");
    while(1);
}
```

WRITE MODE

Writing Register

Write Latch When Enabled

Write Register

Write Latch When Enabled

Write Register

Write Register

`printf(char *fmt, ...)` 处理方式
✓ 中断处理方式

- UART 工作方式为全双工，分发送（TX）和接收（RX）两个独立的方向进行数据传输。

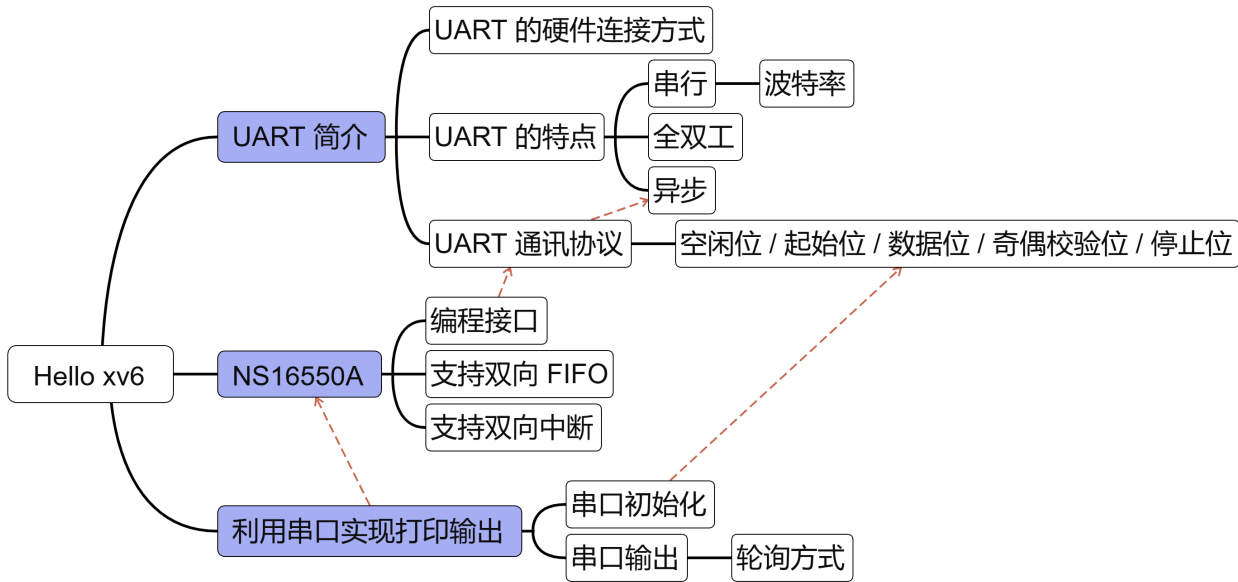
- 对数据的 TX/RX 有两种处理方式：
✓ 中断处理方式

```
u@u-VirtualBox:~/ws/xv6-riscv-step-by-step$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic
1 0
1 1 xv6 kernel is booting
1 1
```

```
WriteReg(THR, c);
```



本章总结



谢谢

